

dataset_prep

June 11, 2026

```
[1]: import json
import re
import warnings
from collections import Counter
from pathlib import Path
```

0.0.1 CONFIG

```
[2]: #    !! CHANGE THESE !!
JSONL_PATH      = "raw_dataset.jsonl"      # <--- UPDATE THIS PATH
TOKENIZER_NAME  = "google/gemma-4-E4B-it"  # Target tokenizer
MAX_CONTEXT     = 8192                     # NEEDS UPDATE
VAL_SPLIT      = 0.10                     # 10% for validation
RANDOM_SEED     = 42
OUT_DIR         = Path("./prepared_data")
OUT_DIR.mkdir(exist_ok=True)
```

0.0.2 Step 1: Setup, Imports, and Load Data

```
[3]: def load_jsonl(filepath: str) -> list:
    """Load a .jsonl file; report parse errors gracefully."""
    records, errors = [], []
    with open(filepath, "r", encoding="utf-8") as f:
        for i, line in enumerate(f, 1):
            line = line.strip()
            if not line:
                continue
            try:
                records.append(json.loads(line))
            except json.JSONDecodeError as e:
                errors.append((i, str(e)))

    print(f" Loaded      : {len(records):,} records")
    if errors:
        print(f" Parse errors on lines: {[e[0] for e in errors]}")
    return records

data = load_jsonl(JSONL_PATH)
```

Loaded : 1,603 records

```
[4]: if data:
    print("\nSample record keys:", list(data[0].keys()))
    print("Conversation turns :", len(data[0].get("conversations", [])))
    print("Metadata keys       :", list(data[0].get("metadata", {}).keys()))
```

Sample record keys: ['conversations', 'metadata']
 Conversation turns : 3
 Metadata keys : ['domain', 'industry', 'level', 'company_size',
 'quality_score', 'q_counts', 'version']

0.0.3 2. Schema Validation

```
[5]: #
    # STEP 2: SCHEMA VALIDATION
    # Every record must have: conversations[system, user, assistant] + metadata
    #

    EXPECTED_ROLES = {"system", "user", "assistant"}

    validation_rows = []
    for i, rec in enumerate(data):
        roles = [c["role"] for c in rec.get("conversations", [])]
        # print(roles)
        validation_rows.append({
            "idx"           : i,
```

```

        "has_conversations": "conversations" in rec,
        "has_metadata"      : "metadata" in rec,
        "has_system"        : "system"   in roles,
        "has_user"          : "user"     in roles,
        "has_assistant"     : "assistant" in roles,
        "extra_roles"       : sorted(set(roles) - EXPECTED_ROLES),
        "role_count"        : len(roles),
    })
validation_rows[0]

```

```

[5]: {'idx': 0,
      'has_conversations': True,
      'has_metadata': True,
      'has_system': True,
      'has_user': True,
      'has_assistant': True,
      'extra_roles': [],
      'role_count': 3}

```

```

[6]: import pandas as pd
      val_df = pd.DataFrame(validation_rows)

```

```

[7]: checks = ["has_conversations", "has_metadata", "has_system", "has_user",
               ↪ "has_assistant"]
      val_df.head()

```

```

[7]:   idx  has_conversations  has_metadata  has_system  has_user  has_assistant  \
0    0                 True             True         True         True         True
1    1                 True             True         True         True         True
2    2                 True             True         True         True         True
3    3                 True             True         True         True         True
4    4                 True             True         True         True         True

      extra_roles  role_count
0              []           3
1              []           3
2              []           3
3              []           3
4              []           3

```

```

[8]: print("SCHEMA VALIDATION RESULTS:")
      print("-" * 30)
      for col in checks:
          fails = int((~val_df[col]).sum())
          icon = " " if fails == 0 else f" {fails} failures"
          print(f"{col:<20}: {icon}")

```

SCHEMA VALIDATION RESULTS:

```

-----
has_conversations      :
has_metadata           :
has_system             :
has_user               :
has_assistant          :

```

```

[9]: # Check for unexpected roles (e.g., 'model' used too early, or typos)
extra = val_df[val_df["extra_roles"].map(len) > 0]
if extra.empty:
    print(f"{'extra_roles':<20}: ")
else:
    print(f"{'extra_roles':<20}:    found in {len(extra)} records →
    ↪{extra['extra_roles'].tolist()[:5]}")

```

```

extra_roles           :

```

```

[10]: # Flag records that don't have exactly 3 turns
wrong_turns = val_df[val_df["role_count"] != 3]
print(f"\nRecords with turn count 3: {len(wrong_turns)}")
if not wrong_turns.empty:
    print(f"Indices of malformed records: {wrong_turns['idx'].tolist()[:10]}")

```

```

Records with turn count 3: 0

```

0.0.4 3. Flatten into a Dataframe

```

[11]: ## converting it into a df so that we can easily filter it, plot it and analyze
    ↪text length

#
# STEP 3: FLATTEN INTO DATAFRAME
#

import numpy as np

def flatten_records(records: list) -> pd.DataFrame:
    rows = []
    for i, rec in enumerate(records):
        convs      = rec.get("conversations", [])
        meta       = rec.get("metadata", {})
        q_counts   = meta.get("q_counts", {})

        # Map out roles to easily extract their text content
        role_map = {c["role"]: c["content"] for c in convs}

        rows.append({
            "idx"           : i,

```

```

        # text fields
        "system"      : role_map.get("system",   ""),
        "user"        : role_map.get("user",     ""),
        "assistant"   : role_map.get("assistant", ""),
        # metadata
        "domain"       : meta.get("domain",      "Unknown"),
        "industry"     : meta.get("industry",    "Unknown"),
        "level"        : meta.get("level",       "Unknown"),
        "company_size" : meta.get("company_size", "Unknown"),
        "quality_score" : meta.get("quality_score", np.nan),
        "version"      : str(meta.get("version", "Unknown")),
        # question-type counts
        "q_technical"  : q_counts.get("[Technical]", 0),
        "q_behavioral" : q_counts.get("[Behavioral]", 0),
        "q_situational" : q_counts.get("[Situational]", 0),
        "q_culture"    : q_counts.get("[Culture]", 0),
        "q_career"     : q_counts.get("[Career]", 0),
    })
    return pd.DataFrame(rows)

df = flatten_records(data)

print(f"DataFrame shape: {df.shape[0]} rows, {df.shape[1]} columns")
print("\nSample rows (first 3):")
display(df[["idx", "domain", "industry", "level", "quality_score"]].head(3))

```

DataFrame shape: 1603 rows, 15 columns

Sample rows (first 3):

	idx	domain \
0	0	Business Analyst
1	1	Product Management
2	2	Machine Learning / AI Engineering

	industry	level \
0	BFSI (Banking, Financial Services, Insurance)	Engineering Manager
1	FinTech / Payments	VP / C-suite
2	Gaming / Metaverse	Engineering Manager

	quality_score
0	100
1	100
2	100

```
[12]: pd.set_option('display.max_colwidth', None)
df.head(1)
```

[12]: idx \
0 0

system \

0 You are an expert technical recruiter and hiring manager with 15+ years of experience across multiple industries. Given a job description, you generate targeted, insight-driven interview questions that help hiring teams assess both technical competence and cultural fit. Your questions are always specific to the role - never generic.

user \

0 Generate 20 interview questions for the following job description:\n\nA leading BFSI (Banking, Financial Services, Insurance) company, dedicated to providing innovative financial solutions, is seeking a skilled Engineering Manager to join its Business Analyst team. With a strong presence in the industry, our company has established itself as a trusted partner for individuals and businesses alike. Our mission is to leverage technology and expertise to deliver exceptional customer experiences and drive business growth.\n\nAs an Engineering Manager in the Business Analyst team, you will own the development and implementation of business solutions, driving significant impact on our company's operations and revenue. You will collaborate with cross-functional teams to identify business needs, design solutions, and oversee their execution. Your expertise will be critical in analyzing complex business problems, identifying opportunities for improvement, and implementing data-driven solutions. You will also mentor and guide junior team members, fostering a culture of innovation and excellence.\n\nKey Responsibilities:\n* Lead the design and development of business solutions using FastAPI 0.110+, ensuring scalability, security, and compliance with industry regulations\n* Collaborate with stakeholders to gather business requirements, using tools like dbt Core for data analysis and visualization\n* Develop and maintain technical documentation using Notion, ensuring that all solutions are well-documented and easily maintainable\n* Work with the engineering team to implement solutions using React 18, ensuring seamless integration with existing systems\n* Analyze business data using Python 3.10+, identifying trends, opportunities, and areas for improvement\n* Develop and manage project plans, resource allocation plans, and timelines using Jira 9.1+, ensuring timely and within-budget delivery of solutions\n* Conduct post-implementation reviews, using tools like Tableau 2022.3+, to assess solution effectiveness and identify areas for improvement\n\nRequired Skills:\n* 5+ years of experience in business analysis, with a strong background in BFSI (Banking, Financial Services, Insurance)\n* Proficiency in FastAPI 0.110+, dbt Core, and React 18\n* Experience with data analysis and visualization tools like Tableau 2022.3+, Power BI 2.104+\n* Strong understanding of industry regulations, including GDPR, HIPAA, and PCI-DSS\n* Excellent communication and project management skills, with experience using Jira 9.1+, Notion, and Slack 4.28+\n* Programming skills in Python 3.10+, with experience using libraries like Pandas 1.5+, NumPy 1.23+\n\nNice-to-Have Skills:\n* Experience with cloud-based technologies like AWS 2022+, Azure 2022+,

or Google Cloud 2022+\n* Knowledge of agile methodologies, including Scrum and Kanban\n* Certification in business analysis, such as CBAP or CCBA\n* Experience with machine learning frameworks like TensorFlow 2.11+ or PyTorch 1.13+\n\nWhat We Offer:\n* Competitive salary and benefits package, including health insurance, retirement plans, and paid time off\n* Opportunities for professional growth and development, including training, mentorship, and education assistance\n* Collaborative and dynamic work environment, with a team of experienced professionals who are passionate about delivering exceptional customer experiences

assistant \

0 1. [Technical] What is your experience with FastAPI 0.110+ and how do you ensure scalability?\n2. [Behavioral] Tell me about a time you had to collaborate with stakeholders to gather business requirements using dbt Core.\n3. [Technical] How do you maintain technical documentation using Notion for business solutions?\n4. [Behavioral] Describe a situation where you had to analyze complex business problems and identify opportunities for improvement.\n5. [Technical] What is your experience with data analysis and visualization tools like Tableau 2022.3+?\n6. [Behavioral] Tell me about a time you had to work with the engineering team to implement solutions using React 18.\n7. [Technical] How do you develop and manage project plans using Jira 9.1+ for timely delivery of solutions?\n8. [Situational] Imagine you are tasked with developing a business solution using Python 3.10+, how would you approach it?\n9. [Behavioral] Tell me about a time you had to conduct post-implementation reviews to assess solution effectiveness.\n10. [Situational] How would you approach a situation where a business solution is not meeting the expected outcome?\n11. [Culture] How do you foster a culture of innovation and excellence in a team, as mentioned in the JD?\n12. [Technical] What is your understanding of industry regulations, including GDPR, HIPAA, and PCI-DSS, in the context of BFSI?\n13. [Behavioral] Describe a situation where you had to communicate technical information to non-technical stakeholders.\n14. [Situational] Imagine you are tasked with mentoring junior team members, how would you approach it?\n15. [Career] What motivates you to take on a role in business analysis, specifically in the BFSI industry?\n16. [Culture] How do you see yourself contributing to a collaborative and dynamic work environment, as described in the JD?\n17. [Technical] What is your experience with cloud-based technologies like AWS 2022+, Azure 2022+, or Google Cloud 2022+?\n18. [Behavioral] Tell me about a time you had to use agile methodologies, including Scrum and Kanban, in a project.\n19. [Situational] How would you approach a situation where a project is behind schedule, and what tools would you use to get it back on track?\n20. [Career] Why do you think this Engineering Manager role is the next step in your career, and how do you see yourself growing in this position?

domain

industry \

0 Business Analyst BFSI (Banking, Financial Services, Insurance)

level

company_size quality_score \

0 Engineering Manager Mid-size Company (200-1,000 employees) 100

	version	q_technical	q_behavioral	q_situational	q_culture	q_career
0	3.1	6	6	4	2	2

```
[13]: df['industry'].value_counts()
```

```
[13]: industry
AgriTech / CleanTech          140
FinTech / Payments            134
Healthcare / MedTech          130
Software / SaaS               121
Manufacturing / Industry 4.0  120
EdTech                        116
Consulting / Professional Services 116
E-commerce / D2C              112
Logistics / Supply Chain      110
Cybersecurity                 109
Media / Entertainment          107
Real Estate / PropTech         104
BFSI (Banking, Financial Services, Insurance) 94
Gaming / Metaverse            90
Name: count, dtype: int64
```

```
[14]: df['domain'].value_counts()
```

```
[14]: domain
Mobile Engineering (iOS / Android)  116
HR / People Operations              106
Finance / FP&A                      99
Machine Learning / AI Engineering  96
Data Science                       96
UI / UX Design                     91
Sales (B2B / Enterprise)            90
Data Engineering                   88
QA / Automation Engineering         83
Backend Engineering                 77
Digital Marketing / Growth          77
DevOps / SRE / Platform Engineering 75
Customer Success                   74
MLOps / LLMOps                     69
Product Management                  68
Frontend Engineering                64
Business Analyst                    60
Full Stack Engineering              60
Cloud Architecture                  57
Technical Program Management        57
```


Name: count, dtype: int64

```
[15]: df.columns
```

```
[15]: Index(['idx', 'system', 'user', 'assistant', 'domain', 'industry', 'level',  
        'company_size', 'quality_score', 'version', 'q_technical',  
        'q_behavioral', 'q_situational', 'q_culture', 'q_career'],  
        dtype='str')
```

0.0.5 4. Token & Length analysis

```
[16]: # 4a. Character & Word counts  
for col in ["system", "user", "assistant"]:  
    df[f"{col}_chars"] = df[col].str.len()  
    df[f"{col}_words"] = df[col].str.split().str.len()  
  
df["total_chars"] = df["system_chars"] + df["user_chars"] +  
    df["assistant_chars"]  
df["total_words"] = df["system_words"] + df["user_words"] +  
    df["assistant_words"]  
  
len_cols = ["system_chars", "user_chars", "assistant_chars", "total_chars",  
            df["total_words"]  
print("Character / Word Statistics:")  
display(df[len_cols].describe().round(1))
```

Character / Word Statistics:

	system_chars	user_chars	assistant_chars	total_chars	total_words
count	1603.0	1603.0	1603.0	1603.0	1603.0
mean	334.0	3094.2	2663.4	6091.6	874.1
std	0.0	235.4	170.3	303.0	42.0
min	334.0	818.0	1902.0	3196.0	462.0
25%	334.0	2955.0	2563.0	5903.0	850.0
50%	334.0	3074.0	2665.0	6089.0	871.0
75%	334.0	3195.5	2772.5	6259.5	893.0
max	334.0	4777.0	3463.0	7771.0	1121.0

```
[17]: from tqdm import tqdm  
  
# 4b. Token counting  
TOKENIZER_AVAILABLE = False  
try:  
    from transformers import AutoTokenizer  
    print(f"\n Loading tokenizer '{TOKENIZER_NAME}' ...")  
  
    # Gemma 3 tokenizer  
    tokenizer = AutoTokenizer.from_pretrained(TOKENIZER_NAME)
```

```

TOKENIZER_AVAILABLE = True
print(f" Tokenizer ready!")

def _count_tokens(row):
    text = row["system"] + "\n" + row["user"] + "\n" + row["assistant"]
    return len(tokenizer.encode(text, add_special_tokens=False))

# Initialize tqdm for pandas to show progress bar
tqdm.pandas(desc="Counting tokens")
df["total_tokens"] = df.progress_apply(_count_tokens, axis=1)

except Exception as e:
    print(f"\n Tokenizer unavailable ({e}).")
    print("Using word-count proxy: tokens  words × 1.35")
    df["total_tokens"] = (df["total_words"] * 1.35).astype(int)

```

```

/home/mindmap/projects/LLM_finetune/.venv/lib/python3.12/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm
[transformers] PyTorch was not found. Models won't be available and only
tokenizers, configuration and file/data utilities can be used.

```

Loading tokenizer 'google/gemma-4-E4B-it' ...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Tokenizer ready!

Counting tokens: 100%| | 1603/1603 [00:05<00:00, 284.21it/s]

```

[18]: # 4c. Token budget summary
over_limit = df[df["total_tokens"] > MAX_CONTEXT]
under_50    = df[df["total_tokens"] < 50]
p95_tokens = int(df["total_tokens"].quantile(0.95))
p99_tokens = int(df["total_tokens"].quantile(0.99))

print(f"\nToken statistics (Context Limit = {MAX_CONTEXT:,}):")
print("-" * 40)
print(f"Min      : {df['total_tokens'].min():>6,}")
print(f"Max      : {df['total_tokens'].max():>6,}")
print(f"Mean     : {df['total_tokens'].mean():>9.1f}")
print(f"Median   : {df['total_tokens'].median():>9.1f}")
print(f"p95     : {p95_tokens:>6,}")
print(f"p99     : {p99_tokens:>6,}")

```

```
print(f"\n Over {MAX_CONTEXT:,} token limit : {len(over_limit):>4} (Will be_
↳dropped/truncated later)")
print(f" Under 50 tokens (Too short): {len(under_50):>4}")
```

Token statistics (Context Limit = 8,192):

```
-----
Min      :    639
Max      :   1,661
Mean     :   1265.1
Median   :   1260.0
p95      :   1,365
p99      :   1,519
```

```
Over 8,192 token limit :    0 (Will be dropped/truncated later)
Under 50 tokens (Too short):    0
```

0.0.6 5. Data Quality Checks

```
[19]: print("DATA QUALITY REPORT")
      print("-" * 30)

      # 5.1 NULL / EMPTY STRINGS
      print("\n5.1 NULL / EMPTY STRINGS:")
      for col in ["system", "user", "assistant"]:
          nulls = df[col].isnull().sum()
          empty = (df[col].astype(str).str.strip() == "").sum()
          print(f" {col:<12}: {nulls} nulls | {empty} empty strings")
```

DATA QUALITY REPORT

```
-----

5.1 NULL / EMPTY STRINGS:
  system      : 0 nulls | 0 empty strings
  user        : 0 nulls | 0 empty strings
  assistant   : 0 nulls | 0 empty strings
```

```
[20]: # 5.2 EXACT DUPLICATES (user + assistant pair)
      dup_mask = df.duplicated(subset=["user", "assistant"], keep=False)
      exact_dups = dup_mask.sum()
      print(f"\n5.2 EXACT DUPLICATE PAIRS: {exact_dups}")
```

5.2 EXACT DUPLICATE PAIRS: 0

```
[21]: # 5.3 NEAR-DUPLICATE USER PROMPTS (first-200-char fingerprint)
      # We convert to lowercase and strip extra whitespace to find highly similar_
      ↳prompts
```

```
df["_user_fp"] = (df["user"].str[:200]
                  .str.lower()
                  .str.replace(r"\s+", " ", regex=True))
near_dups = df.duplicated(subset=["_user_fp"], keep=False).sum()
print(f"\n5.3 NEAR-DUPLICATE USER PROMPTS: {near_dups}")
```

5.3 NEAR-DUPLICATE USER PROMPTS: 744

```
[22]: # 5.4 ASSISTANT FORMAT CHECK (Checking if output starts with numbered lists)
has_numbered = df["assistant"].str.contains(r"^\d+\.", regex=True, na=False)
print(f"\n5.4 ASSISTANT FORMAT CHECK (Expects numbered questions):")
print(f"  Responses WITH numbered Qs      : {has_numbered.sum()}")
print(f"  Responses WITHOUT numbered Qs   : {(~has_numbered).sum()}")
```

5.4 ASSISTANT FORMAT CHECK (Expects numbered questions):

Responses WITH numbered Qs : 1603

Responses WITHOUT numbered Qs : 0

```
[23]: # 5.5 OUTLIER DETECTION (Using Interquartile Range on assistant chars)
q25, q75 = df["assistant_chars"].quantile([0.25, 0.75])
iqr      = q75 - q25
outlier_lo = df[df["assistant_chars"] < q25 - 1.5 * iqr]
outlier_hi = df[df["assistant_chars"] > q75 + 1.5 * iqr]
```

```
[24]: print(f"\n5.5 OUTLIER DETECTION (Assistant Response Length):")
print(f"  IQR range (chars)      : {q25:.0f} - {q75:.0f}")
print(f"  Low-length outliers    : {len(outlier_lo)} (unusually short)")
print(f"  High-length outliers   : {len(outlier_hi)} (unusually long)")
```

5.5 OUTLIER DETECTION (Assistant Response Length):

IQR range (chars) : 2563 - 2772

Low-length outliers : 24 (unusually short)

High-length outliers : 10 (unusually long)

0.0.7 6. Categorical Metadata Columns

```
[25]: print("METADATA DISTRIBUTIONS & CLASS BALANCE")
print("=" * 50)

# 6.1 Categorical Metadata Columns
meta_cols = ["domain", "industry", "level", "company_size"]
for col in meta_cols:
    vc = df[col].value_counts()
    print(f"\n {col.upper()} ({vc.nunique()} unique values):")
    print("-" * 40)
    for val, cnt in vc.items():
```

```
# Create a text-based bar chart using unicode blocks
bar = " " * int(cnt / len(df) * 40)
pct = cnt / len(df) * 100
print(f" {val[:42]:<43} {cnt:>4} {pct:5.1f}% {bar}")
```

METADATA DISTRIBUTIONS & CLASS BALANCE

=====

DOMAIN (16 unique values):

Mobile Engineering (iOS / Android)	116	7.2%
HR / People Operations	106	6.6%
Finance / FP&A	99	6.2%
Machine Learning / AI Engineering	96	6.0%
Data Science	96	6.0%
UI / UX Design	91	5.7%
Sales (B2B / Enterprise)	90	5.6%
Data Engineering	88	5.5%
QA / Automation Engineering	83	5.2%
Backend Engineering	77	4.8%
Digital Marketing / Growth	77	4.8%
DevOps / SRE / Platform Engineering	75	4.7%
Customer Success	74	4.6%
MLOps / LLMops	69	4.3%
Product Management	68	4.2%
Frontend Engineering	64	4.0%
Business Analyst	60	3.7%
Full Stack Engineering	60	3.7%
Cloud Architecture	57	3.6%
Technical Program Management	57	3.6%

INDUSTRY (13 unique values):

AgriTech / CleanTech	140	8.7%
FinTech / Payments	134	8.4%
Healthcare / MedTech	130	8.1%
Software / SaaS	121	7.5%
Manufacturing / Industry 4.0	120	7.5%
EdTech	116	7.2%
Consulting / Professional Services	116	7.2%
E-commerce / D2C	112	7.0%
Logistics / Supply Chain	110	6.9%
Cybersecurity	109	6.8%
Media / Entertainment	107	6.7%
Real Estate / PropTech	104	6.5%
BFSI (Banking, Financial Services, Insuran	94	5.9%
Gaming / Metaverse	90	5.6%

LEVEL (7 unique values):

Junior (0-2 yrs)	283	17.7%
Lead / Principal (8-12 yrs)	271	16.9%
Mid-level (2-5 yrs)	244	15.2%
Director	215	13.4%
Engineering Manager	206	12.9%
VP / C-suite	199	12.4%
Senior (5-8 yrs)	185	11.5%

COMPANY_SIZE (4 unique values):

Early-stage Startup (10-50 employees)	421	26.3%
Growth Startup (50-200 employees)	417	26.0%
Mid-size Company (200-1,000 employees)	389	24.3%
Large Enterprise (1,000+ employees)	376	23.5%

```
[26]: # 6.2 Target Question Type Totals across dataset
print("\n QUESTION TYPE TOTALS ACROSS ENTIRE DATASET:")
print("-" * 40)
q_cols = ["q_technical", "q_behavioral", "q_situational", "q_culture", "q_career"]
labels = ["Technical", "Behavioral", "Situational", "Culture", "Career"]

for col, lbl in zip(q_cols, labels):
    total = int(df[col].sum())
    avg = df[col].mean()
    print(f" {lbl:<12}: total={total:>5,} avg per record={avg:.1f}")
```

QUESTION TYPE TOTALS ACROSS ENTIRE DATASET:

Technical	: total=11,192	avg per record=7.0
Behavioral	: total=8,043	avg per record=5.0
Situational	: total=6,404	avg per record=4.0
Culture	: total=3,216	avg per record=2.0
Career	: total=3,205	avg per record=2.0

0.0.8 7. Visualization (EDA Report)

```
[27]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from tqdm import tqdm
```

```
[28]: # Plot style
plt.rcParams.update({
    "figure.dpi": 130,
    "font.family": "DejaVu Sans",
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.titlesize": 12,
    "axes.titleweight": "bold",
})
PALETTE = ["#4C72B0", "#DD8452", "#55A868", "#C44E52", "#8172B2", "#937860",
    ↪ "#DA8BC3", "#8C8C8C", "#CCB974", "#64B5CD"]
```

```
[29]: # Set up a large grid for our 10 plots
fig = plt.figure(figsize=(22, 24))
fig.suptitle(
    "LLM Fine-Tuning Dataset - EDA Report\n"
    f"Interview Question Generator | {len(df):,} records",
    fontsize=16, fontweight="bold", y=0.99,
)
gs = fig.add_gridspec(4, 3, hspace=0.50, wspace=0.38)
```

<Figure size 2860x3120 with 0 Axes>

```
[31]: # Plot 1: Token count distribution
ax1 = fig.add_subplot(gs[0, 0])
ax1.hist(df["total_tokens"], bins=45, color=PALETTE[0], edgecolor="white", lw=0.
    ↪4)
ax1.axvline(df["total_tokens"].mean(), color="red", lw=1.5, ls="--",
    label=f"Mean: {df['total_tokens'].mean():.0f}")
ax1.axvline(df["total_tokens"].median(), color="orange", lw=1.5, ls=":",
    label=f"Median: {df['total_tokens'].median():.0f}")
ax1.axvline(MAX_CONTEXT, color="black", lw=1.2, ls="-.",
    label=f"Max ctx: {MAX_CONTEXT}")
ax1.set_title("Total Token Distribution")
ax1.set_xlabel("Tokens")
ax1.set_ylabel("Count")
ax1.legend(fontsize=8)

# Plot 2: Assistant response length
ax2 = fig.add_subplot(gs[0, 1])
ax2.hist(df["assistant_chars"], bins=45, color=PALETTE[1], edgecolor="white",
    ↪lw=0.4)
ax2.axvline(df["assistant_chars"].mean(), color="red", lw=1.5, ls="--",
    label=f"Mean: {df['assistant_chars'].mean():.0f}")
ax2.set_title("Assistant Length (chars)")
ax2.set_xlabel("Characters")
ax2.set_ylabel("Count")
```

```

ax2.legend(fontsize=8)

# Plot 3: User prompt length
ax3 = fig.add_subplot(gs[0, 2])
ax3.hist(df["user_chars"], bins=45, color=PALETTE[2], edgecolor="white", lw=0.4)
ax3.axvline(df["user_chars"].mean(), color="red", lw=1.5, ls="--",
            label=f"Mean: {df['user_chars'].mean():.0f}")
ax3.set_title("User Prompt Length (chars)")
ax3.set_xlabel("Characters")
ax3.set_ylabel("Count")
ax3.legend(fontsize=8)

# Plot 4: Top-20 Domain distribution
ax4 = fig.add_subplot(gs[1, :2])
dom_vc = df["domain"].value_counts().head(20)
bars = ax4.barh(dom_vc.index[::-1], dom_vc.values[::-1],
                color=PALETTE[0], alpha=0.85)
for bar, val in zip(bars, dom_vc.values[::-1]):
    ax4.text(bar.get_width() + 0.3, bar.get_y() + bar.get_height() / 2,
             f" {val}", va="center", fontsize=8)
ax4.set_title("Top 20 Domain Distribution")
ax4.set_xlabel("Count")

# Plot 5: Job level pie
ax5 = fig.add_subplot(gs[1, 2])
level_vc = df["level"].value_counts()
wedges, _, autotexts = ax5.pie(
    level_vc.values, labels=None, autopct="%1.1f%%",
    colors=PALETTE[:len(level_vc)], startangle=90, pctdistance=0.78)
for at in autotexts:
    at.set_fontsize(8)
ax5.legend(wedges, level_vc.index, loc="center left",
           bbox_to_anchor=(-0.35, 0.5), fontsize=8)
ax5.set_title("Job Level Distribution")

# Plot 6: Top-20 Industry distribution
ax6 = fig.add_subplot(gs[2, :2])
ind_vc = df["industry"].value_counts().head(20)
bars = ax6.barh(ind_vc.index[::-1], ind_vc.values[::-1],
                color=PALETTE[1], alpha=0.85)
for bar, val in zip(bars, ind_vc.values[::-1]):
    ax6.text(bar.get_width() + 0.3, bar.get_y() + bar.get_height() / 2,
             f" {val}", va="center", fontsize=8)
ax6.set_title("Top 20 Industry Distribution")
ax6.set_xlabel("Count")

# Plot 7: Company size distribution

```



```

ax7 = fig.add_subplot(gs[2, 2])
size_vc = df["company_size"].value_counts()
ax7.bar(range(len(size_vc)), size_vc.values, color=PALETTE[2], alpha=0.85)
ax7.set_xticks(range(len(size_vc)))
ax7.set_xticklabels([s.replace("(", "\n(") for s in size_vc.index], fontsize=8)
ax7.set_title("Company Size Distribution")
ax7.set_ylabel("Count")

# Plot 8: Avg question types per record
ax8 = fig.add_subplot(gs[3, 0])
q_means = df[q_cols].mean()
bars = ax8.bar(labels, q_means.values,
               color=PALETTE[:len(labels)], alpha=0.85)
for bar, val in zip(bars, q_means.values):
    ax8.text(bar.get_x() + bar.get_width() / 2,
             bar.get_height() + 0.02, f"{val:.1f}",
             ha="center", fontsize=8)
ax8.set_title("Avg Question Types / Record")
ax8.set_ylabel("Mean Count")
ax8.tick_params(axis="x", rotation=20)

# Plot 9: Token count by level
ax9 = fig.add_subplot(gs[3, 1])
lvl_order = df["level"].value_counts().index.tolist()
token_by_level = [df[df["level"] == lvl]["total_tokens"].values
                  for lvl in lvl_order]
bp = ax9.boxplot(token_by_level, patch_artist=True,
                 boxprops=dict(facecolor=PALETTE[0], alpha=0.65),
                 medianprops=dict(color="red", lw=2),
                 flierprops=dict(marker=".", markersize=3, alpha=0.4))
ax9.set_xticklabels([l.replace(" / ", "\n") for l in lvl_order], fontsize=8)
ax9.set_title("Token Counts by Job Level")
ax9.set_ylabel("Tokens")

# Plot 10: Data quality summary
ax10 = fig.add_subplot(gs[3, 2])
quality_items = {
    "Valid schema" : int(val_df["has_conversations"].sum()),
    "Has metadata" : int(val_df["has_metadata"].sum()),
    "No exact duplicates" : int(len(df) - exact_dups),
    "Within token limit" : int(len(df) - len(over_limit)),
    "Non-empty assistant" : int((df["assistant"].str.strip() != "").sum()),
}
colors = [PALETTE[2] if v == len(df) else PALETTE[3]
          for v in quality_items.values()]
ax10.barh(list(quality_items.keys()), list(quality_items.values()),
          color=colors, alpha=0.85)

```

```

ax10.axvline(len(df), color="gray", lw=1, ls="--", alpha=0.6,
              label=f"Total = {len(df)}")
ax10.set_title("Data Quality Summary")
ax10.set_xlabel("Record Count")
ax10.legend(fontsize=8)

# Save the final report to disk
report_path = OUT_DIR / "eda_report.png"
fig.savefig(report_path, dpi=150, bbox_inches="tight")
print(f" EDA report generated and saved to: {report_path}")

# Display in notebook
fig.show()

```

EDA report generated and saved to: prepared_data/eda_report.png

```

/tmp/ipykernel_33657/2937046595.py:129: UserWarning: FigureCanvasAgg is non-
interactive, and thus cannot be shown
    fig.show()

```

[]: